cf-kit guide

How the Codeforces setup works, why each part exists, and how to use it.

1. What cf-kit does

Codeforces judges your solution on hidden tests. You only see a few sample tests in the statement. cf-kit gives you a local loop that catches most mistakes before you submit:

1. **Import** the problem and its samples from your browser, with one click.
2. **Write** the solution in a file that already has a fast, correct template.
3. **Judge** it against the samples, with the same compiler flags Codeforces uses.
4. **Stress test** it against a slow, simple solution on hundreds of random inputs. This finds bugs the samples miss.

It works in VS Code or Sublime Text, both with Vim keys. macOS is the tested platform.

2. Install

```
curl -fsSL https://raw.githubusercontent.com/bobikenobi12/cf-kit/main/install.sh
| bash
```

The installer clones the repository to `~/cf-kit`, asks a few yes/no questions, and then:

- checks for GCC, and offers to install it with Homebrew if it is missing;
- configures VS Code and Sublime Text, if you choose them;
- builds the precompiled header (see section 4.3);
- runs the doctor, which checks everything and prints a fix for each problem.

Options: `--yes` accepts all defaults, `--skip-editors` leaves your editors alone. You can run the installer again at any time. It is safe to repeat.

After that, install the **Competitive Companion** extension in your browser. Its project page is github.com/jmerle/competitive-companion.

3. The daily workflow

Keep the listener running in a terminal (`python3 ~/cf-kit/scripts/listen.py`), then:

1. Open a problem on Codeforces and click Competitive Companion. A folder appears: `contests/<contest>/<letter>/`.
2. Open `sol.cpp` and write the solution.
3. Press **Space b** (VS Code) or **Cmd B** (Sublime) to run the samples.

4. Before submitting, press **Space s** to stress test, if you wrote a brute force.

Each problem folder contains:

```
sol.cpp        your solution
brute.cpp      a slow but simple solution, for stress testing
gen.py         a random input generator, for stress testing
problem.json   name, URL and time limit
tests/         the sample inputs (N.in) and expected outputs (N.ans)
build/         compiled programs (created automatically)
```

4. The concepts

4.1 Samples are not enough

The samples in a statement are usually small and easy. A solution can pass them and still fail on large or unusual inputs. That is why cf-kit has two checks: the judge runs the samples, and the stress tester searches for inputs where your solution disagrees with a brute force. A brute force is slow, but simple enough to trust.

4.2 The compiler

Codeforces compiles C++ with GCC on Windows. On macOS the default `g++` is Apple's clang, which does not include `bits/stdc++.h`, the header most competitive programmers use. So cf-kit uses GCC from Homebrew (`g++-16`).

Every compile uses the same flags, defined in `scripts/common.py`:

Flag	Meaning
`-std=gnu++20`	The C++ standard used for most submissions.
`-Wall -Wextra -Wshadow`	Warnings. `-Wshadow` warns when a variable hides another one.
`-O2`	Optimisation, like the real judge. Without it, timings mean little.
`-DLOCAL`	Defines the macro `LOCAL`, so local-only code runs here and is skipped on Codeforces.

Use `ll`, not `long`. On Windows `long` is 32 bits; on macOS it is 64 bits. Code that uses `long` for large numbers can pass here and fail on Codeforces. `ll` means `long long` and is 64 bits everywhere.

4.3 The precompiled header

Including `bits/stdc++.h` makes the compiler read a large amount of standard library code every time. A precompiled header is that code, already processed and saved to disk as `include/bits/stdc++.h.gch`. The compiler loads the saved version instead of reading the source.

It saves about half a second per compile on this Mac. It is optional: without it, everything still works, just a little slower. It must be rebuilt after a compiler upgrade, with `python3 scripts/pch.py`.

4.4 Fast input and output

Contests with a million numbers can time out on slow input. The template fixes this with two lines:

```
ios::sync_with_stdio(false);   // do not sync C++ streams with C stdio
cin.tie(nullptr);             // do not flush cout before each read
```

Also print with `'\n'` instead of `endl`. `endl` flushes the output on every line, which is slow for large output.

4.5 The debug build

A common bug is writing one element past the end of an array. The program often still gives the right answer, because the memory next to the array happens to be free. The release build hides the bug.

The debug build (**Space d**) compiles with AddressSanitizer, UBSan and bounds-checked containers. It stops at the first bad access and prints where it happened:

```
Error: attempt to subscript container with out-of-bounds index 5,
but container only holds 2 elements.
```

Run it when a sample fails for no obvious reason, and before submitting problems that involve indexes or large numbers.

4.6 Stress testing

Stress testing needs three files:

- **brute.cpp**: the simplest correct solution. Speed does not matter.
- **gen.py**: prints one random, valid, small input. It receives a seed as its argument, so every run can be reproduced.
- **sol.cpp**: your real solution.

The stress tester runs seeds 1, 2, 3 and so on, feeds each input to both programs, and compares the outputs. On the first mismatch it stops and saves the input to `stress_fail/input.txt`. Run your solution on that file to reproduce the problem.

Keep the inputs small. The brute force must finish, and most bugs show up on small inputs anyway.

5. Editors

VS Code with Vim

The installer enables the Vim extension and the C/C++ extension. The leader key is **Space**.

Keys	Action
Space b	Run the samples
Space d	Run the samples with the debug build
Space s	Stress test the problem
Space l	Start the listener
j k	Leave insert mode
Cmd Shift B	Run the samples (the default build task, works without Vim)

Sublime Text with Vim

Sublime's built-in Vim mode (Vintage) is enabled. Use **Cmd B** to run the samples. Choose *Tools* > *Build System* > *CF-C++* once, and save the folder as a project to keep the choice. For the debug build, use *Tools* > *Build With* > *Debug*.

The Sublime setup has been checked by generating and validating its files. It has not been run inside the editor by the project.

Snippets

Type the prefix and press Tab.

Prefix	Expands to
cp	The full template
forn / forr	Loop from 0 up to n, or from n-1 down to 0
vin	Read n values into a vector
bs	Binary search for the smallest x where a condition holds
mpow	Modular power
yn	Print YES or NO
dbg	Debug print to stderr, ignored on Codeforces

The snippets are defined once, in `.vscode/cpp.code-snippets`. The installer converts them for Sublime.

6. Contest day

1. Start the listener and leave it running.
2. Click Competitive Companion on each problem, and open `sol.cpp`.
3. For tricky problems, write `brute.cpp` and `gen.py` first.
4. Run the samples after each change. Fix every wrong answer before moving on.
5. Before submitting: run the debug build, then the stress test.
6. Submit `sol.cpp`. The `dbg` lines do nothing on Codeforces.

7. Troubleshooting

Start with `python3 scripts/doctor.py`. It checks the compiler, the header, the port, the editors and the example, and explains each problem.

Problem	Fix
No GCC found	`brew install gcc`. Or set `CP_CXX` to the path of your GCC.
Clicking the extension does nothing	The listener must be running and print *saved ...*. Open `http://127.0.0.1:27121/` in a browser to check it. Another program may be using port 27121.
Compiles are slow	Run `python3 scripts/pch.py`.
Vim keys do nothing in VS Code	Open the repository folder itself, not a subfolder, and check that the Vim extension is enabled.
Your answer is right but the judge says WA	The problem may accept several answers. The judge compares against one stored answer, so check the output by hand.

8. Limits

- **Only samples are saved.** Hidden tests are not available locally, so a passing judge does not guarantee acceptance. Stress testing covers part of that gap.
- **No interactive problems.** The judge runs a program on a fixed input.
- **Approximate comparison.** Output is compared token by token. Floating-point numbers may differ by up to 10^{-6}. Problems with several valid answers need a checker.
- **Linux is not tested yet.** The code is written to work there, but it has not been run on a Linux machine.
- **No debugger is set up.** Use `dbg` and the debug build.

Source: github.com/bobikenobi12/cf-kit. MIT License.